

SUPPLEMENTARY MATERIAL FOR EXPLORING SOCIAL-ACTIVITY NETWORKS FOR SYBIL DETECTION: MODELS AND EFFICIENT ALGORITHMS

Zongyuan Chen¹, Tao Tan², Haoran Gu³, Hong Xie¹, Mingze Zhong⁴,
Xiaoyu Shi⁵, Mingsheng Shang⁵, and John C. S. Lui⁶

¹University of Science and Technology of China ²CCCC Second Highway Consultants Co., Ltd.

³Daqing Oilfield Chongqing Company ⁴Australian Artificial Intelligence Institute

⁵Chinese Academy of Sciences ⁶The Chinese University of Hong Kong

ABSTRACT

This supplementary material contains the complete technical details, theoretical analysis, proofs, and additional experiments supporting the main ICASSP paper.

Index Terms— random walk, social-activity network, Sybil detection, trust propagation, scalable graph representation

1. INTRODUCTION

Fake account (Sybil) detection is important for online social networks (OSNs) because it can protect OSN operators and users from various malicious activities. One mainstream sybil detection approach is the graph-based method [1–5]. Briefly speaking, the graph-based sybil detection approach models an OSN as a graph and exploits the graph structure to identify sybil users, where nodes represent users and edges represent friendship among users. One common assumption of such graph-based sybil detection approach is that sybil users have a limited capability to lure honest users to make friends. In other words, from a graph perspective, this means that there is a sparse cut between honest users and sybil users. Based on this assumption, a variety of sybil detection algorithms were proposed [1–4, 6–11]. However, a number of recent works showed that this sparse cut assumption may not hold in many applications [12–14]. This breaking down of sparse cut assumption leads to low detection accuracy of previous algorithms that built on this assumption [15, 16].

To overcome the non sparse cut challenge, a number of algorithms explored social activities to enhance graph based sybil detection [5, 17–20]. Along this direction, one line of works utilized social activity data to augment the edge of social graphs [17–19, 21]. Besides, Zhang *et al.* [5] initialized the hybrid graph research line, which constructed a two-layer of graph, i.e., friendship graph layer and social activity graph layer. It is shown that the hybrid graph approach achieved a higher accuracy than augmenting the edge of social graph with activity data [5]. Recently, some works applied transformer and graph neural networks to process the hybrid graph for enhancing sybil detection [19, 20]. Though transformer or graph neural network based methods have high expressiveness power, their computational complexity is a major obstacle, especially in the dynamic setting where one needs to update the model from time to time. In contrast, the random walk based method is computationally promising, but existing methods have a limited accuracy in the presence of trapping nodes [5]. Furthermore, existing methods represent each activity as a node, making it not scalable to real-world OSNs with huge volumes of activities. This paper aims to improve the

scalability of random walk based method with respect to the number of activities and mitigate the accuracy limitation caused by trapping nodes. Improving the scalability with respect to social activities is vital for sybil detection in at least in two aspects: (1) the activity contains rich information for identifying sybils and its amount is huge and keep increasing; (2) the behavior of sybils evolves dynamically, and scalable algorithms is highly desirable to adapt to it on the fly.

To facilitate the presentation of our contributions, let us use SybilSocActNet [5], which is one of the representative algorithms, to illustrate the scalability and accuracy issue.

Example 1. Consider a social network with five nodes denoted by $v_1, \dots, v_5$. Two of them are sybil nodes, i.e., v_4 and v_5 . Their friendship, chatting history as well as other activities such as posts, mentioning other user in the posts, etc., are shown in Fig. 1. Node v_3 is a given honest seed node.

Friend pair	Activities
· (v_1, v_2) chat a lot	· v_1 creates a_1, a_2
· (v_1, v_3) rarely chat	· v_3 reposts a_2 as a_3
· (v_2, v_3) chat a lot	· v_3 creates a_4, a_5
· (v_2, v_4) never chat	· v_4 creates a_6
· (v_2, v_5) rarely chat	· v_5 reposts a_6 as a_7
· (v_3, v_4) never chat	· a_1, a_2, a_7 mention v_2
· (v_3, v_5) rarely chat	· a_3, a_4, a_5 mention v_1
· (v_4, v_5) chat a lot	· v_3 likes a_2 , v_5 likes a_6

Fig. 1: Example of friendship and activities.

Fig. 2 shows the social-activity graph constructed by SybilSocActNet based on the activities shown in Fig. 1. Note that SybilSocActNet ignores the activities such as chatting, like or dislike a post, etc. Based on the constructed social activity network and the given honest user node v_3 , SybilSocActNet detects sybil user nodes via a carefully designed random walk algorithm. Two key parameters of the random walk algorithm are: (1) the number of steps that the walker walks between activity and user nodes k ; (2) the number of steps that the walker walks within the activity nodes n . The k and n jointly balance the amount of trust propagated from the trust seed v_3 to the user nodes and activity nodes.

The first limitation of SybilSocActNet is that it is not scalable with respect to activities. Fig. 2 shows that SybilSocActNet represents each activity as a distinct node. In other words, the number of nodes in the graph grows linearly with the number of activities. In real-world OSNs, the number of activities is usually thousands or

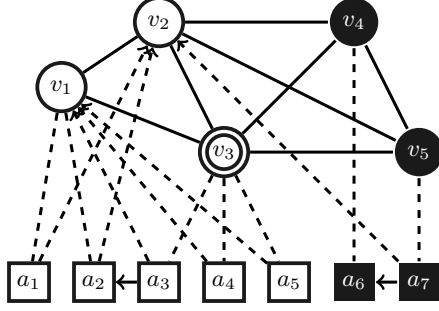


Fig. 2: Social-activity graph constructed by SybilSocActNet.

even more times of the number of user nodes [5]. Furthermore, the number of activities is increasing daily. This makes SybilSocActNet impractical for real-world OSNs.

The second limitation of SybilSocActNet is that its accuracy is sensitive to the parameters k and n . To illustrate, Fig. 3 shows the impact of k and n on the Area Under the Receiver Operating Characteristic Curve (AUC) [16] of SybilSocActNet evaluated on the graph in Fig. 2. Note that AUC is a commonly used metric to quantify sybil detection accuracy and it is 0.5 if one detects sybil users by randomly sorting all the users, i.e., random detection. Fig. 3a shows that as k increases from 0 to 3, the AUC of SybilSocActNet drops from 1 to 0.5, and then remains unchanged. In other words, the accuracy of SybilSocActNet decreases significantly as we increase k slightly. Furthermore, the accuracy is close to random detection when $k \geq 3$, i.e., a very low accuracy. This implies that SybilSocActNet is sensitive to the number of walks on the user-activity graph, making it difficult to tune the parameter k and utilize the user-activity graph. Fig. 3b shows the impact of n on the AUC of SybilSocActNet. When we keep $k = 5$, the AUC of SybilSocActNet remains at 0.5 when n increases from 1 to 10. This shows that selecting different n does not help to improve the accuracy. A detailed analysis of the algorithmic limitations of SybilSocActNet is provided in Appendix A. This analysis also explains why directly applying SybilSocActNet to ScaSAN can lead to low accuracy.

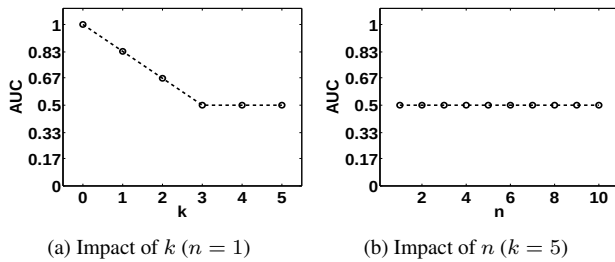


Fig. 3: AUC of SybilSocActNet on the graph in Fig. 2.

Motivated by the above observations, we develop models and algorithms to address the aforementioned scalability and trapping node issue. Our contributions are the following:

- We formulate a hybrid graph called ScaSAN (Scalable Social Activity Network) to represent users and activities in an OSN. ScaSAN aggregates social activities with respect to activity types. The number of nodes is three times of the number of users, instead of scaling linearly with respect to the number of social activities as [5] does.

- We develop SybilScaRepAct (Sybil detection with Scalable Representation of Activities) algorithm to evaluate trust scores for users based on a given tiny set of seed nodes (honest or sybil seeds) and ScaSAN. SybilScaRepAct addresses the trapping node issue by carefully distributing a small amount of trust score of trapping nodes to trusted seed nodes. SybilScaRepAct is interpretable via the random walk paradigm and it uses an iterative algorithm to approximate the trust scores. We prove the convergence rate of this iterative algorithm and derive the number of rounds needed to achieve a given approximation accuracy. The theoretical bounds improve [5] by relying on explicit hyper parameters instead of unknown mixing rate, making it practically usable.
- We conduct extensive experiments on a public dataset from Facebook. Experiments show that SybilScaRepAct has a higher accuracy than SybilSocActNet and other baseline algorithms and reduces the computational time of SybilSocActNet by around 85%.

2. RELATED WORK

2.1. Feature-based Sybil Detection

Feature-based sybil detection methods use feature engineering to extract features of each node from user's activities, profile, etc. The research line can be categorized into unsupervised detection [22–25] and supervised detection [12, 26]. Recent advancement of this research line are as follows. Xia *et al.* [27] analyzed the patterns of friendship requests, and identified dozens of new features to detect malicious accounts. Morstatter *et al.* [28] analyzed the behavioral patterns of sybils using a dataset from the game World of Tanks. [29, 30] used LLMs to detect social bots via text analysis. Jiang *et al.* [31] combined both static feature like address and dynamic feature like proportion of forwarding to detection social bots. Liu *et al.* [32] leveraged the similarity of contents among users to detect social bots. Our work complements this line of sybil detection research by proposing a graph-based sybil detection algorithm.

2.2. Graph-based Sybil Detection

Graph-based sybil detection method exploits graph pattern to detect sybil nodes and random walk is a main stream approach. Early works along this research line were built on the sparse cut assumption [1–4, 6–11]. Furutani *et al.* [33] revealed theoretical understandings of random walk based and belief propagation based sybil detection algorithms through the lens of graph signal processing. The cut is shown to be non sparse in a variety of applications [12–14]. A number of works studied this issue [16, 34, 35]. Various side information was exploited, i.e., detecting victims, detecting honest users who is a friend of sybils, to enhance sybil detection [34, 35]. Notably, Effendy *et al.* [16] utilized the strength of friendship to pruned attack edges. Cao *et al.* [36] and Xue *et al.* [37] proposed a friend accepting and rejecting mechanism to enhance sybil detection. Gao *et al.* [38] proposed SybilFuse, which contains a local phase and global phase, where the local phase computes local trust scores for nodes and edges, and the global phase propagates local trust scores over the global network. Liu *et al.* [39] proposed Accou2vec, which learns the vector representation of accounts via community walk. Yang *et al.* [40] proposed a novel structural balance model to study signed social networks, which revealed important insights on user behavior for supplementing sybil detection. Hammi *et al.* [41] studied the

compliance of several sybil detection approaches with respect to cooperative intelligent transportation system standards, and provided important recommendations for further study.

A number of recent works enhanced graph based sybil detection via social activity data. Yang *et. al* [42] enhanced graph based sybil detection via friendship request data. A signed interaction graph is formulated, where a directed edge with weight $-1/1$ indicates that a user rejects/accepts a friendship request. Sybils were detected via graph embedding. Yuan *et. al* [43] enhanced graph based sybil detection via the registration data. A measurement study on a dataset from WeChat (the largest online social network in China) was conducted to characterize the registration pattern. Based on the pattern, a graph is constructed to detect sybils. A follow up of this work [17] further observed a cluster pattern of fake accounts with respect to IP and phone numbers. A registration graph was designed to capture the identified pattern, and a community detection method is utilized to detect sybils. Qu *et. al* [18] proposed Mush, which uses a Multistimuli Hawkes process to analyze review patterns for sybil detection. Lingam *et. al* [21] augmented the Twitter network graph via the behavior data. Behavioral similarity was summarized from tweet-content, shared URL, interest, and social interactions to augment the edge of social graph. Liu *et. al* [44] proposed a hierarchical information compression approach to summarize the data of social networks. The proposed coarse-grained and fine-grained stages enable high-quality knowledge representation from social networks, which in turn would enhance sybil detection. Zhang *et. al.* [5] proposed a hybrid graph approach, which consists of a two layers of graph, i.e., friendship graph layer and social activity graph layer. It is showed that this approach is superior over augmenting the graph edge with social activity data. Shin *et. al* [45] proposed RICC, which improves the adversarial robustness of Sybil detection. Li *et. al* [19] developed a transformer based method for sybil detection. Huang *et. al* [20] developed a graph neural network based method for sybil detection. Efficient neural architecture search methods like [46] would enhance the accuracy and computational efficiency of these methods significantly. Though transformer or graph neural network based methods have high expressiveness power, their computational complexity is a major obstacle, especially in the dynamic setting where one needs to update the model from time to time. In contrast, the random walk based method is computationally promising, but existing methods have a limited accuracy in the presence of trapping nodes [5]. Furthermore, existing methods represent each activity as a node, making it not scalable to real-world OSNs with huge volumes of activities. This paper aims to address the scalability and trapping node issue.

Lastly, one important aspect ignored by current sybil detection methods is the user behavior pattern hidden in the communication network layer. We believe that data and techniques from the communication networks such as data routing [47], mobility pattern mining [48], and networking environment change detection [49] would provide valuable complement for sybil detection.

3. THE MODEL

3.1. The ScaSAN Graph Model

We formulate a weighted “mixed” graph model to represent users and activities in a social network. The mixed graph contains both weighted undirected edges and weighted directed edges. We formally denote it as:

$$\mathcal{G} \triangleq (\mathcal{V}, \mathcal{A}, \tilde{\mathcal{A}}, \mathbf{E}, \mathbf{C}, \mathbf{F}, \mathbf{S}, \mathbf{M}, \mathbf{L}),$$

where $\mathcal{V}$, $\mathcal{A}$ and $\tilde{\mathcal{A}}$ are node sets characterizing users and activities, and $\mathbf{E}, \mathbf{C}, \mathbf{F}, \mathbf{S}, \mathbf{M}, \mathbf{L}$ are weight matrices summarizing friendship and frequency of activities. We call $\mathcal{G}$ the ScaSAN.

Nodes of the graph. The node set $\mathcal{V} \triangleq \{v_1, \dots, v_N\}$ denotes the set of users, where each node represents one user in a social network and $N \in \mathbb{N}_+$. The node set $\mathcal{A} \triangleq \{A_1, \dots, A_N\}$ models the activity of creating contents. In particular, A_i represents an aggregation of all the contents created by user $v_i \in \mathcal{V}$ such as posts on Facebook, tweets on Twitter, etc. The node set $\tilde{\mathcal{A}} \triangleq \{a_1, \dots, a_N\}$ models the activity of forwarding contents. In particular, a_i represents an aggregation of all the contents forwarded by user $v_i \in \mathcal{V}$ such as reposts on Facebook, retweets on Twitter, etc. To simplify notations, A_i (or a_i) serves as dummy node if user $v_i \in \mathcal{V}$ does not create (or forward) any contents. For example, in Fig. 4 we have $N = 5$ users, $\mathcal{V} = \{v_1, \dots, v_5\}$, $\mathcal{A} = \{A_1, \dots, A_5\}$ and $\tilde{\mathcal{A}} = \{a_1, \dots, a_5\}$. Furthermore, a_1, A_2, a_2, a_4 and A_5 are dummy nodes and we omit them in the Fig. 4 to make it clean.

Edges of the graph. The weight matrix $\mathbf{E} \triangleq [e_{u,v} : u, v \in \mathcal{V}] \in \mathbb{R}_{\geq 0}^{|\mathcal{V}| \times |\mathcal{V}|}$ summarizes the friendship and “undirected” activities like chatting and conversation between users. Note that $e_{u,v} > 0$ if and only if u and v are mutual friends. Furthermore, we consider a symmetric friendship graph, i.e., $e_{u,v} = e_{v,u}$. A larger weight $e_{u,v}$ implies that user u and v have more frequent “undirected” activities. For example, in Fig. 4, $e_{v_1, v_2} = 3$ represents that user v_1 and v_2 are mutual friends and they chat a lot, i.e., $e_{v_3, v_5} = 2$ represents that user v_3 and v_5 are mutual friends and they chat rarely, and $e_{v_2, v_4} = 1$ represents that user v_2 and v_4 are mutual friends but never chat.

The weight matrix $\mathbf{C} \triangleq [c_{v,A} : v \in \mathcal{V}, A \in \mathcal{A}] \in \mathbb{R}_{\geq 0}^{|\mathcal{V}| \times |\mathcal{A}|}$ summarizes the number of contents created by users. Note that $c_{v,A} > 0$ if and only if user v creates at least one content, e.g., posts on Facebook or tweets on Twitter. A larger weight $c_{v,A}$ implies that user v has created more contents. For example, in Fig. 4, $c_{v_1, A_1} = 2$ means that user v_1 creates 2 contents, and $c_{v_5, A_5} = 0$ means user v_5 never creates content and the dummy edge (v_5, A_5) is omitted.

The weight matrix $\mathbf{F} \triangleq [f_{v,a} : v \in \mathcal{V}, a \in \tilde{\mathcal{A}}] \in \mathbb{R}_{\geq 0}^{|\mathcal{V}| \times |\tilde{\mathcal{A}}|}$ summarizes the number of contents forwarded by users. Note that $f_{v,a} > 0$ if and only if user v forwards at least one content such as reposts in Facebook and retweets in Twitter. A larger weight $f_{v,a}$ implies that user v has forwarded more contents. For example, in Fig. 4, $f_{v_1, a_1} = 0$ means user v_1 never forwarded any contents, and $f_{v_3, a_3} = 1$ means user v_3 forwarded one content.

The weight matrix $\mathbf{S} \triangleq [s_{a,b} : a \in \tilde{\mathcal{A}}, b \in \mathcal{A} \cup \tilde{\mathcal{A}}] \in \mathbb{R}_{\geq 0}^{|\tilde{\mathcal{A}}| \times |\mathcal{A} \cup \tilde{\mathcal{A}}|}$ tracks how contents are forwarded. Note that $s_{a,b} > 0$ if and only if the user associated with activity a forwards at least one activity of the user associated with activity b . A larger weight $s_{a,b}$ implies a larger number of such forwarding of contents. For example, in Fig. 4, $s_{a_3, A_1} = 1$ means that user v_3 forwards one content created by user v_1 . The same contents can be forwarded by many other users.

The weight matrix $\mathbf{M} \triangleq [m_{b,v} : b \in \mathcal{A} \cup \tilde{\mathcal{A}}, v \in \mathcal{V}] \in \mathbb{R}_{\geq 0}^{|\mathcal{A} \cup \tilde{\mathcal{A}}| \times |\mathcal{V}|}$ summarizes the mentioning behavior of users. Note that $m_{b,v} > 0$ if and only if the user associated with activity b mentions user v in activity b at least once. A larger weight $m_{b,v}$ implies a larger number of such mentioning of users. For example, in Fig. 4, $m_{A_1, v_2} = 2$ means that in the contents created by user v_1 , the user v_1 mentions user v_2 twice.

The weight matrix $\mathbf{L} \triangleq [l_{v,b} : v \in \mathcal{V}, b \in \mathcal{A} \cup \tilde{\mathcal{A}}] \in \mathbb{R}_{\geq 0}^{|\mathcal{V}| \times |\mathcal{A} \cup \tilde{\mathcal{A}}|}$ summarizes the activities such as like or comment-

ing a content created by other users. Note that $l_{v,b} > 0$ if and only if user v likes or comments activity b at least once. A larger weight $l_{v,b}$ implies a larger number of like or commenting behavior. For example, in Fig. 4, $l_{v_3,A_1} = 1$ means user v_3 likes or comments on an activity created by user v_1 .

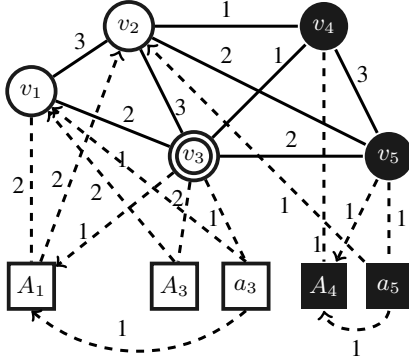


Fig. 4: ScaSAN constructed from the OSN in Fig. 1.

Lemma 1. *The ScaSAN has at most $3N$ nodes. The space complexity of the ScaSAN is at most $3N + \Phi(\mathbf{E}) + \Phi(\mathbf{F}) + \Phi(\mathbf{C}) + \Phi(\mathbf{S}) + \Phi(\mathbf{M}) + \Phi(\mathbf{L})$, where $\Phi(\cdot)$ denotes the number of nonzero entries of a input matrix.*

Lemma 1 states the space complexity of the ScaSAN graph. Note that N is the number of users. This implies that number of nodes of our model is three times of the number of users. The number of nodes of SybilSocActNet [5] equals the number of users plus the number activities. The number of nodes of other works such as [17, 21] equals the number of users, since they uses activity data to augment friendship edge. We do not follow the friendship edge augmenting approach, because it was showed that this approach is inferior to the hybrid graph approach [5].

Discussions on the model: Our model is generic in capturing activities in real-world OSNs and it is not restricted any instance of OSNs. First, it provides a complete characterization of the activities. More specifically, there are two types of activities, i.e., “undirected” ones and “directed” ones. The undirected activities are captured by the weighted links between users. From a user’s point of view, the directed activities are characterized by contents, which are of two types: the content created by the user itself modeled by A_n ; and the content created by other users modeled by a_n . Note that the content can be a post, a comment, etc. Second, there are three typical types of activities regarding contents and all of them are captured in our model, i.e., forwarding or sharing captured by $s_{a,b}$, mentioning captured by $m_{b,v}$ and like or commenting captured by $l_{v,b}$. We differentiate these three types of activities because they reflect different levels of trust and they are not redundant in terms of graph structure. For the ease of presentation, we do not enumerate all possible types of activities. Some activities such chatting groups, commenting behavior in more complex ways, etc., are not explicitly modeled. Our model is flexible to capture them. For example, a chatting group can be decomposed into interactions pairs and then captured via the social network weight. Commenting behavior in more complex ways can be decomposed into pairs of creating a comment and mentioning a comment, and then captured via the activity graph. One potential limitation of our model is in handling dynamic user behavior. The behavior of sybil users may evolve across time and sybil users may

even game the detection algorithm [45]. One potential way to handle this issue is by adding a discounting factor in activity aggregation, paying more attention to recent activities than old activities. Exploring sophisticated methods is an interesting future direction.

3.2. The Sybil Attack Model

Sybil users launch friendship attacks and activity attacks.

Friendship attacks. Consider the case that the users can be classified into honest or sybil users, i.e., $\mathcal{V} = \mathcal{V}_h \cup \mathcal{V}_s$ and $\mathcal{V}_h \cap \mathcal{V}_s = \emptyset$, where $\mathcal{V}_h$ ($\mathcal{V}_s$) denotes a set of honest (sybil) users.

Definition 1. *An edge (u, v) , $\forall u, v \in \mathcal{V}$ is a friendship attack edge, if u, v satisfies $e_{u,v} > 0$ and $\mathbb{1}_{\{u,v \in \mathcal{V}_h\}} + \mathbb{1}_{\{u,v \in \mathcal{V}_s\}} = 0$.*

Namely, a friendship attack edge is an edge that lies between honest users and sybil users. For example, in Fig. 4, we have $\mathcal{V}_h = \{v_1, v_2, v_3\}$, $\mathcal{V}_s = \{v_4, v_5\}$ and the set of friendship attack edge is $\{(v_2, v_5), (v_2, v_4), (v_3, v_4), (v_3, v_5)\}$. Let V_{FA} denote the volume of friend attacks:

$$V_{FA} \triangleq \sum_{u \in \mathcal{V}_h} \sum_{v \in \mathcal{V}_s} e_{u,v}.$$

For example, in Fig. 4, we have $V_{FA} = 1 + 1 + 2 + 2 = 6$. We consider the practical setting that the volume of friendship attacks V_{FA} is not restricted to be small. Note that this significantly relaxes the assumption of the previous graph-based sybil attack model [1, 2, 6–11].

Activity attacks. Denote the creating contents activity and forwarding contents activity nodes associated with all honest users respectively as

$$\begin{aligned} \mathcal{A}_h &\triangleq \{A_h | c_{v_h, A_h} > 0, v_h \in \mathcal{V}_h\}, \\ \tilde{\mathcal{A}}_h &\triangleq \{a_h | f_{v_h, a_h} > 0, v_h \in \mathcal{V}_h\}. \end{aligned}$$

Denote the creating contents activity and forwarding contents activity nodes associated with all sybil users respectively as

$$\begin{aligned} \mathcal{A}_s &\triangleq \{A_s | c_{v_s, A_s} > 0, v_s \in \mathcal{V}_s\}, \\ \tilde{\mathcal{A}}_s &\triangleq \{a_s | f_{v_s, a_s} > 0, v_s \in \mathcal{V}_s\}. \end{aligned}$$

For example, in Fig. 4, we have $\mathcal{A}_h = \{A_1, A_3\}$, $\mathcal{A}_s = \{A_4\}$, $\tilde{\mathcal{A}}_h = \{a_3\}$ and $\tilde{\mathcal{A}}_s = \{a_5\}$. The following definition defines incoming activity attacks.

Definition 2. *Edge (b, v) is an incoming activity attack edge, if it satisfies $m_{b,v} > 0$, $b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h$ and $v \in \mathcal{V}_s$. Edge (a, b) is an incoming activity attack edge, if $s_{a,b} > 0$, $a \in \tilde{\mathcal{A}}_h$ and $b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s$. Edge (v, b) is an incoming activity attack edge, if $l_{v,b} > 0$, $v \in \mathcal{V}_h$ and $b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s$.*

Namely, each incoming activity attack edge is initiated from the honest users or honest activities to sybil users or sybil activities. Let V_{IAA} denote the volume of incoming activity attack edges:

$$\begin{aligned} V_{IAA} &\triangleq \sum_{b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h} \sum_{v \in \mathcal{V}_s} m_{b,v} \\ &+ \sum_{a \in \tilde{\mathcal{A}}_h} \sum_{b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s} s_{a,b} \\ &+ \sum_{v \in \mathcal{V}_h} \sum_{b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s} l_{v,b}. \end{aligned}$$

For example, in Fig. 4, we have $V_{IAA} = 0 + 0 + 0 = 0$. The smaller the V_{IAA} is, the less likely that honest nodes initiating interactions to sybil nodes. Let ξ denotes the selectiveness of honest nodes initiating interactions to sybil nodes $\xi \triangleq V_{IAA}/V_{HH}$, where V_{HH} is defined as the volume of interactions among honest nodes:

$$\begin{aligned} V_{HH} \triangleq & \sum_{b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h} \sum_{v \in \mathcal{V}_h} m_{b,v} \\ & + \sum_{a \in \tilde{\mathcal{A}}_h} \sum_{b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h} s_{a,b} \\ & + \sum_{v \in \mathcal{V}_h} \sum_{b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h} l_{v,b}. \end{aligned}$$

For example, in Fig. 4, we have $V_{HH} = 5 + 1 + 1 = 7$ and $\xi = 0$. The selectiveness ξ reflects the difficulty of distinguishing sybils from honest users using activity data, i.e., it becomes more difficult to distinguish them using activity data when ξ increases. When ξ is sufficiently large, the activity data may have a negative impact on sybil detection, i.e., misleading the algorithm to classify sybils as honest users. In the design of our sybil detection algorithm, we do not added any restriction or assume any prior knowledge on ξ . We will use numerical experiments to study the impact of ξ on the sybil detection accuracy of our algorithm.

The following definition defines outgoing activity attacks.

Definition 3. Edge (b, v) is an outgoing activity attack edge, if $m_{b,v} > 0$, $b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s$ and $v \in \mathcal{V}_h$. Edge (a, b) is an outgoing activity attack edge, if $s_{a,b} > 0$, $a \in \tilde{\mathcal{A}}_s$ and $b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h$. Edge (v, b) is an outgoing activity attack edge, if $l_{v,b} > 0$, $v \in \mathcal{V}_s$ and $b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h$.

Namely, each outgoing activity attack edge is initiated from the sybil users or sybil activities to the honest users or honest activities. Let V_{OAA} denote the volume of outgoing activity attack edges:

$$\begin{aligned} V_{OAA} \triangleq & \sum_{b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s} \sum_{v \in \mathcal{V}_h} m_{b,v} \\ & + \sum_{a \in \tilde{\mathcal{A}}_s} \sum_{b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h} s_{a,b} \\ & + \sum_{v \in \mathcal{V}_s} \sum_{b \in \mathcal{A}_h \cup \tilde{\mathcal{A}}_h} l_{v,b}. \end{aligned}$$

For example, in Fig. 4, we have $V_{OAA} = 1 + 0 + 0 = 1$. The larger the V_{OAA} is, the more aggressive the sybil nodes are in initiating interactions to honest users. Let ζ denote the aggressiveness of sybil nodes initiating interactions to honest nodes: $\zeta \triangleq V_{OAA}/V_{SS}$, where V_{SS} is defined as the volume of interactions among sybil nodes:

$$\begin{aligned} V_{SS} \triangleq & \sum_{b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s} \sum_{v \in \mathcal{V}_s} m_{b,v} \\ & + \sum_{a \in \tilde{\mathcal{A}}_s} \sum_{b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s} s_{a,b} \\ & + \sum_{v \in \mathcal{V}_s} \sum_{b \in \mathcal{A}_s \cup \tilde{\mathcal{A}}_s} l_{v,b}. \end{aligned}$$

For example, in Fig. 4, we have $V_{SS} = 0 + 1 + 1 = 2$ and $\zeta = 0.5$. We consider the case that the value of ζ and V_{SS} can be arbitrarily large, i.e., sybils can initiate arbitrary number of interactions to honest users or among themselves so as to reduce the chance to be detected.

Assumption 1. A set of seed users $\mathcal{S} \subseteq \mathcal{V}$ is given and it satisfies $|\mathcal{S}| \geq 1$. Furthermore, if $|\{v | v \in \mathcal{S} \cap \mathcal{V}_h\}| > 0$, then $|\mathcal{S}_{ha}| > 0$, where

$$\begin{aligned} \mathcal{S}_{ha} \triangleq & \{A \in \mathcal{A} \mid c_{v,A} > 0, v \in \mathcal{S} \cap \mathcal{V}_h\} \\ & \cup \{a \in \tilde{\mathcal{A}} \mid f_{v,a} > 0, v \in \mathcal{S} \cap \mathcal{V}_h\}. \end{aligned}$$

denotes the honest seed activity set. If $|\{v | v \in \mathcal{S} \cap \mathcal{V}_s\}| > 0$, then $|\mathcal{S}_{sa}| > 0$, where

$$\begin{aligned} \mathcal{S}_{sa} \triangleq & \{A \in \mathcal{A} \mid c_{v,A} > 0, v \in \mathcal{S} \cap \mathcal{V}_s\} \\ & \cup \{a \in \tilde{\mathcal{A}} \mid f_{v,a} > 0, v \in \mathcal{S} \cap \mathcal{V}_s\}. \end{aligned}$$

denotes the sybil seed activity set.

Assumption 1 captures that if there are sybil (or honest) seed users, these seeds have at least one activity. Let $\mathcal{S} \subset \mathcal{V}$ denote a set of seed users. Each user in the seed set $\mathcal{S}$ can be either a trusted user of a sybil, but with a given label. It is conventional in the sybil detection literature that $\mathcal{S}$ is given [1, 2, 5].

Problem 1. Given a set of seed users $\mathcal{S}$ satisfying Assumption 1, design algorithms to evaluate the trust score of nodes.

Note that the trust score is a generic metric and it enables various methods to detect sybils such as ranking users based on the trust score, treat nodes with trust score below a threshold as sybil nodes, etc. Our objective is to address the scalability and trapping node issue of existing methods like [5].

4. ALGORITHM DESIGN

We develop a SybilScaRepAct algorithm to evaluate trust scores of users based on a tiny set of seed nodes (honest or sybil seeds). SybilScaRepAct uses an iterative algorithm to compute the trust score. We prove the convergence rate of SybilScaRepAct and derive the number of rounds needed to achieve a given accuracy.

4.1. General Algorithmic Framework

For presentation brevity, we only focus on how to use trust seeds to propagate trust. When all seeds are sybil nodes, the distrust associated with sybil seeds can be propagated in a reversed manner as that of trust propagation. When seeds is composed of both honest and sybil nodes, one can propagate trust and distrust separately, and them combine the trust score and distrust score of nodes to detect sybils. We use the “random walk framework” to propagate trust from honest seeds to other nodes. We choose the random walk framework because it is computationally efficient and highly interpretable.

Random walk oracle. We first define a random walk oracle, which will be useful in the design of SybilScaRepAct.

Definition 4. Define *RW-Oracle* as a random walk oracle, which is composed of the following three functions:

- *RW.TranMatrix*($\mathcal{G}, \mathcal{S}$) constructs a transition matrix $\mathbf{P}$, which satisfies that the Markov chain associated with $\mathbf{P}$ is ergodic.
- *RW.StaDis*($\mathbf{P}, \mathbf{s}, \epsilon$) returns $\hat{\mathbf{s}}$ satisfying $\|\hat{\mathbf{s}} - \mathbf{s}^*\|_2 \leq \epsilon$, where $\mathbf{s}^*$ denotes the stationary distribution of $\mathbf{P}$.
- *RW.Norm*($\mathbf{E}, \hat{\mathbf{s}}$) normalizes the $\hat{\mathbf{s}}$ using friend graph $\mathbf{E}$.

Note that $\mathbf{P}$ denotes the transition probability matrix of the random walk, which will be constructed later. The $\mathbf{s}$ denotes the initial trust score vector, whose formula is stated in Equation (1). The $\hat{\mathbf{s}}/\mathbf{s}^*$ denotes the approximated/ground-truth trust score. The ϵ denotes the acceptable approximation error of the trust score. The random walk oracle `RW-oracle` provides three functions to assist trust score computing. We defer the implementation details of `RW-oracle` to Section 4.2, and here we focus on their functionality and applying it to design SybilScaRepAct. The `RW.TranMatrix` function constructs the transition matrix $\mathbf{P}$ of a random walk from the graph $\mathcal{G}$ and honest seeds $\mathcal{S}$. The constructed transition matrix $\mathbf{P}$ satisfies that the Markov chain associated with it is ergodic, i.e., the Markov chain converges to a unique stationary distribution. The `RW.StaDis`($\mathbf{P}, \mathbf{s}, \epsilon$) computes an estimation of the stationary distribution of $\mathbf{P}$ with accuracy guarantee. Note that computing the exact stationary distribution is computationally expensive, as $\mathbf{P}$ is of a large scale. We approximate the stationary distribution for the purpose of saving computational cost. The function `RW.Norm` uses friend graph $\mathbf{E}$ to normalize the estimated stationary distribution. This normalization aims to eliminate the bias caused by the weighted degree of nodes.

Algorithmic framework. Based on the random walk oracle defined in Definition 4, Algorithm 1 outlines our algorithmic framework to evaluate the trust score of nodes, given a set of trust seeds. To run Algorithm 1, one needs to specify graph $\mathcal{G}$, trust seeds set $\mathcal{S}$, initial distribution $\mathbf{s} \triangleq (s_i : i \in \{1, \dots, 3N\})$, estimation accuracy $\epsilon \in (0, 1)$. The initial distribution of the random walk is set as follows:

$$s_i = \begin{cases} 1/|\mathcal{S}|, & \text{if } 1 \leq i \leq N \text{ and } v_i \in \mathcal{S}, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

The interpretation of this initial distribution is as follows. The total amount of trust score for honest seed nodes are 1, and each seed user node equally shares the trust score. The output is the trust score for all user nodes $\hat{\mathbf{s}}$. First, we input the graph $\mathcal{G}$ and trust seeds $\mathcal{S}$ into `RW.TranMatrix` to calculate transition matrix $\mathbf{P}$ of a random walk. Then, input the transition matrix, initial distribution and estimation accuracy into `RW.StaDis` to compute an estimation of the stationary distribution. Finally, we input the estimated stationary distribution and the friend graph $\mathbf{E}$ into `RW.Norm` to normalize the estimated stationary distribution and return it as the trust score of users.

Algorithm 1: SybiSRA

Input: $\mathcal{G}$, trust seeds $\mathcal{S}$, $\mathbf{s}$, ϵ

Output: The trust score for all nodes

- 1 $\mathbf{P} \leftarrow \text{RW.TranMatrix}(\mathcal{G}, \mathcal{S})$
 - 2 $\hat{\mathbf{s}} \leftarrow \text{RW.StaDis}(\mathbf{P}, \mathbf{s}, \epsilon)$
 - 3 $\tilde{\mathbf{s}} \leftarrow \text{RW.Norm}(\mathbf{E}, \hat{\mathbf{s}})$
 - 4 **return** $\tilde{\mathbf{s}}$
-

4.2. The Random Walk Oracle

Algorithm 2 implements a random walk oracle `RW-oracle` that satisfies the property stated in Definition 4. The `RW-oracle` contains three functions.

RW.TranMatrix function. Recall that there are in total $|\mathcal{V}| + |\mathcal{A}| + |\tilde{\mathcal{A}}| = 3N$ nodes in the graph $\mathcal{G}$. Though there may be some dummy nodes in $\mathcal{A}$ or $\tilde{\mathcal{A}}$, we consider a $3N \times 3N$ transition matrix to

simplify presentation, where we set the corresponding row and column of the transition matrix to $\mathbf{0}$. Formally, we define the transition matrix as

$$\mathbf{P} \triangleq [p_{i,j} : i, j \in \{1, \dots, 3N\}] \in \mathbb{R}^{3N \times 3N}.$$

We index the element of them such that in each column (and each row), the indexes from 1 to N correspond to users, i.e., index i corresponds to v_i . The indexes from $(N + 1)$ to $2N$ correspond to activities of creating contents, i.e., index i corresponds to A_{i-N} . The indexes from $(2N + 1)$ to $3N$ correspond to activities of forwarding contents, i.e., index j corresponds to a_{j-2N} . There are three types of nodes, we next divide the design of the transition matrix of a random walk into three parts accordingly.

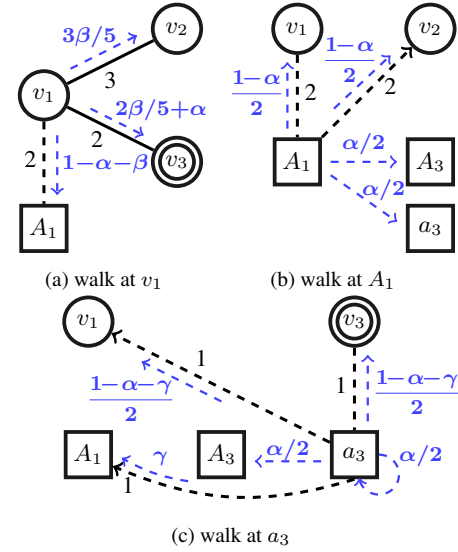


Fig. 5: Random walks in example.

Firstly, consider the case that the walker is at a user node v_i . The high level idea of trust propagation is: α portion of the trust score of v_i is distributed to seed user nodes, β portion of it is distributed to neighbors of v_i , and $1 - \alpha - \beta$ portion of it is distributed to activities, where $0 < \alpha < 1, 0 < \beta < 1, \alpha + \beta < 1$. More specifically, with probability $\alpha/|\mathcal{S}|$ the walker jumps to one of the seed user nodes, i.e., distributing $\alpha/|\mathcal{S}|$ portion of trust score to each seed user. The walker jumps to each neighbor with probability proportional to the weight of the friendship edge, i.e., distributing $\beta e_{v_i, v_j} / \sum_{k=1}^N e_{v_i, v_k}$ portion of trust score to its neighbor v_j . This transition probability is summarized in line 3 of Algorithm 2. For example, consider the graph in Fig. 4 with one seed user v_3 , Fig. 5a illustrates the transition probability on v_1 : $\mathbb{P}[v_3|v_1] = \beta \frac{2}{2+3} + \alpha \times 1 = 2\beta/5 + \alpha$.

We use two parameters λ_1 and λ_2 to facilitate the distribution of trust score across activity edges, where $\lambda_2 > \lambda_1 > 1$. In particular, in the total weight of activity edges associated with v_i , we use $\lambda_2 > \lambda_1 > 1$ to weight activities edges capturing that the edge of creating a content has the highest weight followed by that of forwarding a content, and that of like a content has the smallest weight. The parameters λ_2 and λ_1 enable one to incorporate prior knowledge on different types of activities. For example, one can increase λ_2 to incorporate that one is more certain about that the edge of creating a content is trustworthy. Note that SybilSocAct-Net does not have this feature. If v_i has at least one activity, the

Algorithm 2: The Random Walk Oracle RW-Oracle

```

1 Procedure RW.TranMatrix( $\mathcal{G}, \mathcal{S}$ ):
2   for  $\forall v_i \in \mathcal{V}$  do
3      $\mathbb{P}[v_j|v_i] = \beta e_{v_i, v_j} / \sum_{k=1}^N e_{v_i, v_k} + \alpha s_j$ 
4      $q = \sum_{k=1}^N (\lambda_2 c_{v_i, A_k} + l_{v_i, A_k}) + (\lambda_1 f_{v_i, a_k} + l_{v_i, a_k})$ 
5     if  $q > 0$  then
6        $\mathbb{P}[A_j|v_i] = (1 - \alpha - \beta)(\lambda_2 c_{v_i, A_j} + l_{v_i, A_j})/q$ 
7        $\mathbb{P}[a_j|v_i] = (1 - \alpha - \beta)(\lambda_1 f_{v_i, a_j} + l_{v_i, a_j})/q$ 
8     else
9        $\mathbb{P}[A_j|v_i] = (1 - \alpha - \beta)\mathbb{1}_{\{A_j \in \mathcal{S}_{ha}\}}/|\mathcal{S}_{ha}|$ 
10       $\mathbb{P}[a_j|v_i] = (1 - \alpha - \beta)\mathbb{1}_{\{a_j \in \mathcal{S}_{ha}\}}/|\mathcal{S}_{ha}|$ 
11   for  $\forall A_i \in \mathcal{A}$  do
12      $\mathbb{P}[v_j|A_i] = (1 - \alpha) \frac{c_{v_j, A_i} + m_{A_i, v_j}}{\sum_{k=1}^N c_{v_k, A_i} + m_{A_i, v_k}}$ 
13      $\mathbb{P}[A_j|A_i] = \alpha \mathbb{1}_{\{A_j \in \mathcal{S}_{ha}\}}/|\mathcal{S}_{ha}|$ 
14      $\mathbb{P}[a_j|A_i] = \alpha \mathbb{1}_{\{a_j \in \mathcal{S}_{ha}\}}/|\mathcal{S}_{ha}|$ 
15   for  $\forall a_i \in \tilde{\mathcal{A}}$  do
16      $\mathbb{P}[v_j|a_i] = (1 - \alpha - \gamma) \frac{f_{v_j, a_i} + m_{a_i, v_j}}{\sum_{k=1}^N f_{v_k, a_i} + m_{a_i, v_k}}$ 
17      $\mathbb{P}[A_j|a_i] = \gamma \frac{s_{a_i, A_j}}{\sum_{k=1}^N s_{a_i, A_k} + s_{a_i, A_k}} + \alpha \frac{\mathbb{1}_{\{A_j \in \mathcal{S}_{ha}\}}}{|\mathcal{S}_{ha}|}$ 
18      $\mathbb{P}[a_j|a_i] = \gamma \frac{s_{a_i, a_j}}{\sum_{k=1}^N s_{a_i, a_k} + s_{a_i, A_k}} + \alpha \frac{\mathbb{1}_{\{a_j \in \mathcal{S}_{ha}\}}}{|\mathcal{S}_{ha}|}$ 
19   Set the value for each element of transition matrix:
20   
$$p_{ij} \leftarrow \begin{cases} \mathbb{P}[v_j|v_i], & \text{if } 1 \leq i, j \leq N, \\ \mathbb{P}[A_j \% N | v_i], & \text{if } 1 \leq i \leq N, N+1 \leq j \leq 2N, \\ \mathbb{P}[a_j \% N | v_i], & \text{if } 1 \leq i \leq N, 2N+1 \leq j \leq 3N, \\ \mathbb{P}[v_j | A_i \% N], & \text{if } N+1 \leq i \leq 2N, 1 \leq j \leq N, \\ \mathbb{P}[A_j \% N | A_i \% N], & \text{if } N+1 \leq i \leq 2N, N+1 \leq j \leq 2N, \\ \mathbb{P}[a_j \% N | A_i \% N], & \text{if } N+1 \leq i \leq 2N, 2N+1 \leq j \leq 3N, \\ \mathbb{P}[v_j | a_i \% N], & \text{if } 2N+1 \leq i \leq 3N, 1 \leq j \leq 3N, \\ \mathbb{P}[A_j \% N | a_i \% N], & \text{if } 2N+1 \leq i \leq 3N, N+1 \leq j \leq 2N, \\ \mathbb{P}[a_j \% N | a_i \% N], & \text{if } 2N+1 \leq i \leq 3N, 2N+1 \leq j \leq 3N \end{cases}$$

21   return  $\mathbf{P} = [p_{ij} : i, j \in \{1, \dots, 3N\}]$ 
22 Procedure RW.StaDis( $\mathbf{P}, \mathbf{s}, \epsilon$ ):
23    $\hat{\mathbf{s}}^{(0)} \leftarrow \mathbf{s}$ , evaluate  $t(\epsilon)$  based on (2)
24   for  $\tau = 0, \dots, t(\epsilon) - 1$  do
25      $\hat{\mathbf{s}}^{(\tau+1)} = \hat{\mathbf{s}}^{(\tau)} \mathbf{P}$ 
26   return  $\hat{\mathbf{s}}^{(t(\epsilon)+1)}$ 
27 Procedure RW.Norm( $\mathbf{E}, \hat{\mathbf{s}}$ ):
28    $\tilde{s}_i = \hat{s}_i / \sum_{k=1}^N e_{v_k, v_i}, \forall i = 1, \dots, |\mathcal{V}|.$ 
29   return  $\tilde{\mathbf{s}}$ 

```

walker jumps to its activity nodes with probability proportional to the weight of activity edges weighted by λ_1 and λ_2 , i.e., distributing $(1 - \alpha - \beta)(\lambda_2 c_{v_i, A_j} + l_{v_i, A_j})/q$ portion of trust score to A_j and $(1 - \alpha - \beta)(\lambda_1 f_{v_i, a_j} + l_{v_i, a_j})/q$ portion of trust score to a_j . This transition probability is summarized in line 6-7 of Algorithm 2. For example, consider the graph in Fig. 4, Fig. 5a illustrates the transition probability on v_1 : $\mathbb{P}[A_1|v_1] = (1 - \alpha - \beta) \frac{\lambda_2 \times 2}{\lambda_2 \times 2} = 1 - \alpha - \beta$.

If v_i does not have any activity node, i.e., $q = 0$, the walker jumps to one of the seed activity nodes with equal probability, i.e., distributing $(1 - \alpha - \beta)/|\mathcal{S}_{ha}|$ portion of trust score to each seed activity, where $\mathcal{S}_{ha}$ denotes the set of activity nodes associated with honest seed nodes. This transition probability is summarized in line 9-10 of Algorithm 2.

Secondly, consider the case that the walker is at an activity node A_i . The high level idea of trust propagation is: α portion of the trust score of A_i is distributed to seed activity nodes, $1 - \alpha$ portion of it is distributed to user nodes associated with A_i . More specifically, with probability α the walker jumps to one of the seed activity nodes with equal probability, i.e., distributing $\alpha/|\mathcal{S}_{ha}|$ portion of trust score to each seed activity. This transition probability is summarized in line 13-14 of Algorithm 2. For example, consider the graph in Fig. 4 with two seed activities A_3 and a_3 , Fig. 5b illustrates the transition probability on A_1 : $\mathbb{P}[A_3|A_1] = \alpha * 1/(1 + 1) = \alpha/2$.

The walker jumps to each user node associated with A_i with probability proportional to the weight of an edge, i.e., distributing $(1 - \alpha)(c_{v_j, A_i} + m_{A_i, v_j})/(\sum_{k=1}^N c_{v_k, A_i} + m_{A_i, v_k})$ portion of trust score to user v_j . This transition probability is summarized in line 12 of Algorithm 2. For example, consider the graph in Fig. 4, Fig. 5b illustrates the transition probability on A_1 : $\mathbb{P}[v_1|A_1] = (1 - \alpha) \frac{2}{2+2} = (1 - \alpha)/2$.

Lastly, consider the case that the walker is at an activity node a_i . The high level idea of trust propagation is: α portion of the trust score of a_i is distributed to seed activity nodes, γ portion of it is distributed to activities associated with activity a_i and $1 - \alpha - \gamma$ portion of it is distributed to user nodes, where $0 < \alpha < 1, 0 < \gamma < 1, \alpha + \gamma < 1$. The walker jumps to each seed activity node with equal probability, i.e., distributing $\alpha/|\mathcal{S}_{ha}|$ portion of trust score to each seed activity node. The walker jumps each activity associated with a_i with probability proportional to the weight of the edge, i.e., distributing $\gamma s_{a_i, A_j} / \sum_{k=1}^N s_{a_i, a_k} + s_{a_i, A_k}$ portion of trust score to activity A_j and distributing $\gamma s_{a_i, a_j} / \sum_{k=1}^N s_{a_i, a_k} + s_{a_i, A_k}$ portion of trust score to activity a_j . This transition probability is summarized in line 17-18 of Algorithm 2. For example, consider the graph in Fig. 4 with two seed activities A_3 and a_3 , Fig. 5c illustrates the transition probability on a_3 : $\mathbb{P}[A_1|a_3] = \gamma \frac{0}{1} + \alpha \frac{0}{1+1} = \gamma$, $\mathbb{P}[A_3|a_3] = \gamma \frac{0}{1} + \alpha \frac{1}{1+1} = \alpha/2$.

The walker jumps to the users associated with a_i with probability proportional to the weight of an edge, i.e., distributing $(1 - \alpha - \gamma)(f_{v_j, a_i} + m_{a_i, v_j}) / \sum_{k=1}^N f_{v_k, a_i} + m_{a_i, v_k}$ portion of trust score to user v_j . This transition probability is summarized in line 16 of Algorithm 2. For example, consider the graph in Fig. 4, Fig. 5c illustrates the transition probability on a_3 : $\mathbb{P}[v_1|a_3] = (1 - \alpha - \gamma) \frac{1}{1+1} = (1 - \alpha - \gamma)/2$.

The following theorem states the convergence property of the Markov chain associated with the transition matrix produced by `RW.TranMatrix`.

Theorem 1. Suppose that the friendship graph $(\mathcal{V}, \mathbf{E})$ is connected, $0 < \alpha < 1, 0 < \alpha + \beta < 1, 0 < \alpha + \gamma < 1$. The `RW.TranMatrix` function implemented in Algorithm 2 satisfies the property stated in Definition 4. The `RW.TranMatrix` function implemented in Algorithm 2 has a computational complexity of $O(N^2)$ and a space

complexity of $O(N^2)$.

Theorem 1 states sufficient conditions under which the `RW.TranMatrix` function implemented in Algorithm 2 satisfies the property stated in Definition 4. It guides us to select appropriate parameters α, β and γ for SybilScaRepAct. It also states that both the computational complexity and the space complexity of the `RW.TranMatrix` function implemented in Algorithm 2 are at most quadratic in the number of users.

RW.StaDis function. To assist the design of the `RW.StaDis` function, we first state a theorem to characterize the convergence rate of the Markov chain associated with the transition matrix produced by `RW.TranMatrix`.

Theorem 2. The transition matrix produced by `RW.TranMatrix` satisfies that

$$\|\mathbf{s}^{(0)} \mathbf{P}^{2t} - \mathbf{s}^*\|_1 \leq \left[1 - \min\{1 - \alpha - \gamma, \alpha + \beta\} \frac{\alpha}{|\mathcal{S}|} \right]^t \|\mathbf{s}^{(0)} - \mathbf{s}^*\|_1,$$

where $\mathbf{s}^{(0)} \in [0, 1]^{3N}$ denotes an initial distribution satisfying $\|\mathbf{s}^{(0)}\|_1 = 1$.

Theorem 2 states the mixing rate of the Markov chain associated with the transition matrix produced by `RW.TranMatrix` is geometric. It implies the following corollary to assist us estimate the stationary distribution of this Markov chain. Intuitively, given the initial trust score vector $\mathbf{s}^{(0)}$, after one round of propagation, the trust score vector becomes $\mathbf{s}^{(0)} \mathbf{P}$. After one more round of propagation, the trust score vector becomes $\mathbf{s}^{(0)} \mathbf{P} \times \mathbf{P} = \mathbf{s}^{(0)} \mathbf{P}^2$. Doing this recursively for t rounds, the trust score vector becomes $\mathbf{s}^{(0)} \mathbf{P}^t$. Eventually, $\mathbf{s}^{(0)} \mathbf{P}^t$ converges to the stationary distribution vector $\mathbf{s}^*$ of the Markov chain associated with the transition matrix $\mathbf{P}$. In general, the convergence speed is affected by the distance between the initial trust score and the stationary distribution vector $\mathbf{s}^*$. The $\mathbf{s}^*$ is unknown in advance, but the prior knowledge is that the trust seeds should have a high score. One can apply this prior knowledge to set the initial trust score (do the contrary for sybil seeds). Let $t(\epsilon)$ denote the number of iterations to guarantee that the trust score has an approximation error of at most ϵ .

Corollary 1. By setting

$$t(\epsilon) = 2 \left\lceil \frac{\ln(0.5\epsilon)}{\ln(1 - \min\{1 - \alpha - \gamma, \alpha + \beta\} \alpha / |\mathcal{S}|)} \right\rceil, \quad (2)$$

we can have $\|\mathbf{s}_0 \mathbf{P}^{t(\epsilon)} - \mathbf{s}^*\|_1 \leq \epsilon$.

Corollary 1 states the number of rounds needed to simulate the Markov chain such that the landing probability is close to the stationary distribution with a given accuracy. Based on this we implement `RW.StaDis` function as follows. We first initialize the initial distribution and calculate the value of $t(\epsilon)$ based on (2) (line 22). Then we use $t(\epsilon)$ iterations to approximate the stationary distribution (line 23-24). Corollary 1 implies that the `RW.StaDis` function implemented in Algorithm 2 satisfies the property stated in Definition 4. The following theorem states both the computational complexity and the space complexity of `RW.StaDis`.

Theorem 3. The `RW.StaDis` function implemented in Algorithm 2 has a computational complexity of $O(N^2 t(\epsilon))$ and a space complexity of $O(N^2)$.

Theorem 3 states that both the computational complexity and space complexity of `RW.StaDis` implemented in Algorithm 2 are at most quadratic in the number of users.

RW.Norm function. Note that the stationary distribution of our random walk is biased toward the degree of nodes. Namely, if we use the stationary distribution to detect sybils, honest users with few friends may be misclassified as sybil nodes, while sybil users with many friends may be misclassified as honest users. To eliminate this bias we normalize the stationary distribution with the cumulative friendship degree of user nodes (line 27).

Theorem 4. *The `RW.Norm` function implemented in Algorithm 2 has a computational complexity of $O(N^2)$ and a space complexity of $O(N^2)$.*

Theorem 4 states that both the computational complexity and the space complexity of the `RW.Norm` function implemented in Algorithm 2 are at most quadratic in the number of users.

Putting them together. Putting the computational complexity and the space complexity of `RW.TranMatrix`, `RW.TranMatrix` and `RW.Norm` together, we state the computational and space complexity of Algo. 1 as follows.

Theorem 5. *Given `RW.TranMatrix`, `RW.TranMatrix` and `RW.Norm` implemented in Algorithm 2, The computational complexity of Algorithm 1 is $O(N^2t(\epsilon))$ and the space complexity of Algorithm 1 is $O(N^2)$.*

Theorem 5 states that both the computational complexity and the space complexity of Algorithm 1 are at most quadratic in the number of users.

5. EXPERIMENTS

In this section, we conduct extensive experiments to evaluate the accuracy of SybilScaRepAct. Experiments further confirm the high accuracy as well as high computational efficiency of SybilScaRepAct over a variety of baselines.

5.1. Experimental Setting

The experiment target is to answer the following three questions: **RQ1:** *What's the impact of sybil attack strength on detection accuracy?* **RQ2:** *What's the impact of hyper parameters on detection accuracy?* **RQ3:** *What's the running time improvement?*

Datasets. Following the experiment method of previous works [1, 2, 5, 16], we use a public dataset from Facebook [50] as honest users and activities, and we synthesize sybil users and sybil activities. The public dataset from Facebook [50] contains 63,891 users, 817,090 reciprocal following edges and 876,993 wall posts among users' wall. Each posts has a time stamp. Note that the preferable datasets should contain both friendship links and activity data. Since the activity data may have privacy concerns, it is rarely published. To the best of our efforts, the facebook dataset is the only one we get. One potential limitation of using only the facebook dataset is that it does not contain sybil label. We address this limitation by systematically synthesize the sybil attack for a comprehensive evaluation. This data synthesize approach is commonly used in previous works [5, 16, 34, 35].

Data processing. We map the wall posts into four types of activities, i.e., activity of creating contents, activity of forwarding contents, activity of like contents, activity of mentioning a user. More specifically, we use the following rule to map the activities. Note that each

wall post has a time stamp and the ID of the poster. For each wall, we rank the posts according the time stamp. In the ranking list, we aggregate each continuum sub-sequence with the same poster ID as a block. Then we rank the block according to the time stamp and index it according to its rank. In each block, the first post is mapped as an activity A_i (if the index of the block is an odd number) or a_i (if the index of the block is an even number) of its poster denoted by v_i . The second post in this block (in case there is more than one post in a block) is mapped as a mentioning edge from the afore mapped activity A_i or a_i to the poster of the block ranked just before this block. The remaining posts in this block is mapped as like edges. Each post is mapped as one like edge, which likes the activity of previous blocks. We process the remaining post one by one by chronological, while associating the liked activity to each post in a reversed chronological order. If the number of previous activities is smaller than the number of like edges, we stop after the last activity is used up. We repeat the above process for the remaining posts in this block. We continue this process until all posts in this block are processed. We rank users based on their ID and use sliding window method to select a block of 1000 users, whose average number of activities is close to the average number of activities over the whole dataset. We duplicate the nodes and edges within these 1000 users as sybil region (attack edged will be generated in parameter setting part). Then, in the honest region, there are 63,891 users and 590,262 activities. In the sybil region, there are 1,000 sybils and 6,187 activities. Finally, we locate the largest component of the honest user nodes and the sybil user nodes respectively. For each user node that does not belong to the largest connected component, we connect it to a node (selected uniformly at random) in the largest connected component. Note that SybilSocActNet does not utilize like edges, for fair comparison, we do not use like edges in our experiment, i.e., $V_{HH} = 429,547$, $V_{SS} = 318$. On average, each nodes created 9 activities, so we randomly choose ten users who created 9 activities as seed nodes.

Performance metric. Following previous works [3, 4, 16], we use *Area Under the Receiver Operating Characteristic Curve*(AUC) to quantify the accuracy of sybil detection algorithms. Let $\mathbf{q} \triangleq [q(v_i) : i = 1 \dots, |\mathcal{V}|]$ denote the trust vector for users. Formally, the AUC is defined as:

$$AUC(\mathbf{q}) \triangleq \frac{\sum_{v_i \in \mathcal{V}_h, v_j \in \mathcal{V}_s} (\mathbb{1}_{q(v_i) > q(v_j)} + 0.5 \times \mathbb{1}_{q(v_i) = q(v_j)})}{|\mathcal{V}_h| \times |\mathcal{V}_s|}.$$

Higher AUC indicates that a sybil detection algorithm has a higher accuracy. The case $AUC = 1.0$ (100%) indicates that a sybil detection algorithm makes no errors. $AUC = 0.5$ can be achieved by randomly ranking honest users and sybils.

Comparison baselines. We compare our SybilScaRepAct with the following two algorithms: (1) **SybilSocActNet**: the latest sybil detection algorithm that utilizes activities and it is shown to outperform a variety of recent sybil detection algorithms [5], i.e., SybilRank [6], SR-W [?], SScar [4], SWalk [3], etc.; (2) **SybilFriendship**: a variant of SybilScaRepAct that utilizes friendship edges only. We are aware of many sybil detection algorithms but we only compare with the aforementioned two algorithms due to the following reasons. The are a larger amount of sybil detection algorithms utilizing only friendship edge and relying on the assumption of a sparse cut between honest users and sybil users. Here we drop this assumption and we explore activities to improve sybil detection accuracy. Thus, intuitively our algorithm will outperform these algorithms and comparing to them in a setting without the sparse cut assumption is unfair and may mislead readers. SybilSocActNet outperforms other activity based methods, because it does compress activities. Thus

comparing SybilSocActNet can reveal fundamental insights on the efficiency of our SybilScaRepAct. Comparing to SybilFriendship can reveal insights on the capability of our SybilScaRepAct in utilizing the activities. To the best of our knowledge, SybilSocActNet [5] is the state-of-the-art random walk based algorithm using social-activity network. We are aware of some recent works that use transformer or graph neural network to address this problem [19,20]. However, the transformer and graph neural network is computationally demanding compared to random walk based method. Comparing to them does not yield meaningful insights, since the scalability is one of our main objectives.

Parameter setting. We set the parameters of SybilSocActNet as that in [5]. Unless we vary them explicitly, we consider the following default parameters for our SybilScaRepAct. We set the probability $\beta = 0.04$ (SybilSocActNet has parameter similar to β and it is set to around 0.04), set the probability $\alpha = 0.1$, the probability $\gamma = 0.425$ (SybilSocActNet has parameter similar to γ and it is set to around 0.425). We set the estimation accuracy to $\epsilon = 0.001$. We also set $\lambda_2 = 3$ and $\lambda_1 = 2$. In the implementation, we detect the convergence using the same method as [5] for fair comparison. In previous works [51], the ξ is set as $\xi \approx 4.2 \times 10^{-5}$. This selection of ξ is reasonable because the number of sybil users is extremely small compared to the number of honest users, resulting that the volume of interactions from honest users to sybil users is extremely small compared to the interactions among honest users. We thus set the selectiveness of honest nodes in initiating interactions to sybil nodes as $\xi = 0.00002$ by default, and we will vary ξ to study its impact on our proposed algorithm systematically. For each selection of ξ , we randomly add $\lceil \xi V_{HH} \rceil$ activity edges from honest region to sybil region, where each activity edge can correspond to mentioning behavior, forwarding behavior or like behavior with equal probability. We set the aggressiveness of sybil nodes in initiating interactions to honest nodes as $\zeta = 0.27$. The users of each attack edge is random selected from honest users and sybil users. The number of friendship attack edge is 3200, each attacked honest user is selected from honest users with the number of activities no larger than the median of the number of activities of the whole honest user population uniformly at random. Note that all friendship edges are unweighted.

Running environment. The system specification is: Intel(R) Xeon(R) Platinum 8275CL CPU @ 3.00GHz Memory 182GB.

5.2. Impact of Attack Strength on Detection Accuracy

We evaluate the impact of sybil attack strength on the AUC of algorithms in comparison to answer **RQ1**.

Impact of friendship attack. Fig. 6a shows the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship when the volume of friendship attacks V_{FA} varies from 1600 to 12800. From Fig. 6a, one can observe that the AUC curve of SybilScaRepAct lies above that of SybilSocActNet and SybilFriendship. In other words, SybilScaRepAct has higher accuracy than SybilSocActNet and SybilFriendship with various V_{FA} . The reason is that SybilFriendship does not use activity data, and the activity data in this setting contains useful information to detect sybil. Though SybilSocActNet uses activity data, but it suffers from the trapping node issue. Furthermore, when V_{FA} increases from 1600 to 12800 (three times of the number of friendship edges among sybil region), the AUC of SybilScaRepAct drops from around 0.9 to around 0.8, i.e., a slight drop, while that of SybilSocActNet and SybilFriendship drops from around 0.75 to around 0.5, i.e., a larger drop than SybilScaRepAct. Namely, our SybilScaRepAct is more robust to friendship attacks. The reason for the drop of AUC is that more friendship attack links make the sybil

nodes less distinguishable.

Impact of incoming activity attack. Fig. 6b shows the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship when selectiveness of honest nodes initiating interactions to sybil nodes ξ varies from 0.00002 to 0.00128. From Fig. 6b, one can observe that the AUC curve of SybilScaRepAct lies above that of SybilSocActNet. In other words, SybilScaRepAct has a higher accuracy than SybilSocActNet. The reason is that has the SybilSocActNet trapping node issue and SybilScaRepAct fixes them. Furthermore, as ξ increases from 0.00002 to 0.00128, the AUC of SybilScaRepAct decreases from around 0.9 to around 0.55 and meets AUC curve of SybilFriendship at ξ . The AUC of SybilSocActNet decreases from around 0.7 to around 0.5 and meet AUC curve of SybilFriendship at ξ . The AUC curves of SybilFriendship are flat because this algorithm detect sybil using friendship edges only. When the number of incoming activity attack edges is large, SybilScaRepAct and SybilSocActNet has a smaller AUC than SybilFriendship. The reason is that a large amount of incoming activity attack edges would make sybil nodes indistinguishable from honest nodes, and thus the activity data would mislead the algorithm.

Summary. SybilScaRepAct outperforms both SybilSocActNet and SybilFriendship under different strengths of sybil attack, except when the strength of incoming activity attack is strong. Under this exception, SybilFriendship outperforms SybilScaRepAct due to that the activity data misleads SybilScaRepAct. *The impacts of outgoing activity attacks, seed selection, and activity-edge weights are reported in Appendix B.*

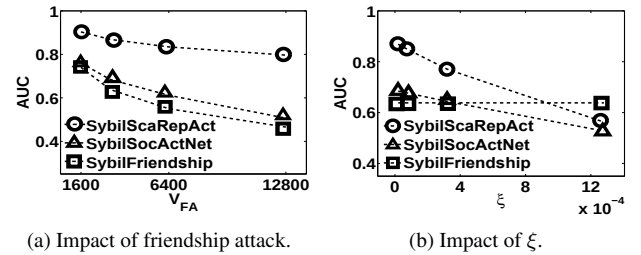


Fig. 6: Impact of Attack Strength on Sybil Detection Accuracy.

5.3. Impact of Hyper Parameters on Detection Accuracy

We evaluate the impact of the hyper parameters of SybilScaRepAct on the AUC to answer **RQ2**.

Impact of α . Fig. 7a shows the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship when α varies from 0.01 to 0.2. From Fig. 7a, one can observe that the AUC curve of SybilScaRepAct lies above that of SybilSocActNet and SybilFriendship. In other words, SybilScaRepAct has higher accuracy than SybilSocActNet and SybilFriendship. Furthermore, the AUC of SybilScaRepAct increases from around 0.7 to around 0.9 when α increases from 0.01 to 0.1, while it becomes flat when α further increases from 0.1 to 0.2. This implies that SybilScaRepAct can attain a very high accuracy by setting an appropriately small α (e.g., 0.1), and this selection is not difficult as the AUC curve is flat for quite a large interval of α , i.e., $\alpha \in [0.1, 0.2]$. The AUC curves of SybilSocActNet and SybilFriendship are flat because these two algorithms do not have the parameter α . Note that α denotes the portion of trust score distributed to seed nodes. The reason for the increase of AUC is that distributing more trust to the seed would improve the trust propagation, when α

is moderately small. It becomes flat because in the moderate range distributing more does not make significant difference.

Impact of β . Fig. 7b shows the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship when β varies from 0.01 to 0.8. From Fig. 7b, one can observe that the AUC curve of SybilScaRepAct lies above that of SybilSocActNet and SybilFriendship. In other words, SybilScaRepAct has a higher accuracy than SybilSocActNet and SybilFriendship. Furthermore, the AUC of SybilScaRepAct decreases from around 0.9 to around 0.7 when β increases from 0.01 to 0.8. This implies that the SybilScaRepAct explores more friendship, the accuracy of sybil detection drops. SybilScaRepAct can attain a very high accuracy when β is small enough (e.g., $\beta \in [0.01, 0.4]$). The AUC curves of SybilSocActNet and SybilFriendship are flat because these two algorithms do not have the parameter β . Note that β denotes the portion of trust score distributed to friends. Since we consider the setting of a large amount of friendship attack edges. Distributing more trust score to friends, would make the SybilScaRepAct rely more on the friendship graph, leading to drop on the accuracy.

Summary. Distributing more trust score to seed nodes would improve the AUC of SybilScaRepAct slightly, when the fraction is moderate. Distributing more trust to neighbors leads AUC drop, when the strength of friendship attack is strong.

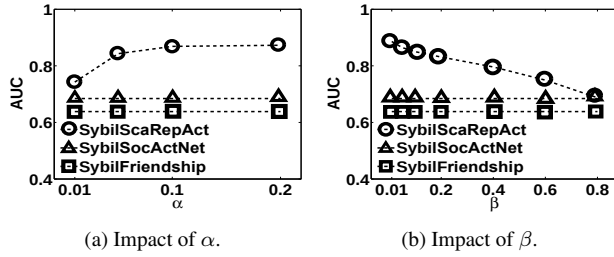


Fig. 7: Impact of Hyper Parameters on Detection Accuracy.

5.4. Evaluating Running Time

We evaluate the running time of SybilScaRepAct and SybilSocActNet to answer **RQ3**. We do not compare to SybilFriendship, since it only uses friendship graph, leading to meaningless comparison. Fig. 8 shows the amount of time needed to compute the trust score, where we vary the estimation error from $\epsilon = 0.002$ to 0.0005. From Fig. 8, one can observe that SybilSocActNet takes at least six times the time of our SybilScaRepAct. In other words, SybilScaRepAct achieves a 6x speedup over SybilSocActNet. This shows the high computational efficiency of our SybilScaRepAct algorithm.

Summary. SybilScaRepAct achieves a 6x speedup over SybilSocActNet.

6. PROOF TO LEMMAS AND THEOREMS

Proof of Theorem 1: It suffices to show that the Markov chain associated with P is recurrent and its period is one. The inequality $0 < \alpha + \beta < 1$ implies that $\beta < 1 - \alpha$. Observe that $0 < \alpha < 1$. Thus we have that $0 < \beta < 1$. As the friendship (V, E) is connected, then it follows that for each pair of nodes (v_i, v_j) , there is a nonzero probability that v_i and v_j communicates. As $0 < \alpha + \beta < 1$, for each activity node A_j , it communicates with the user node who creates it. As each user node communicates with

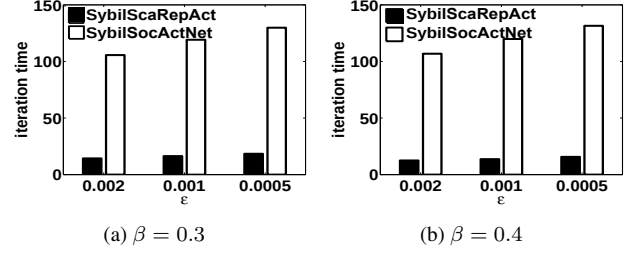


Fig. 8: Iteration time (s) for different estimation error.

each other user nodes, each user node communicates with each activity node A_j . With a similar argument, we can show that each user node communicates with each activity node a_j . Then we conclude that the Markov chain associated with P is recurrent. Note that each user node jumps backs to an honest node with nonzero probability. Thus, the period of the Markov chain associated with P is one. This proof is then complete. The computational complexity and the space complexity are concluded by noting that the graph G has at most $3N$ nodes. ■

Proof of Theorem 2: Suppose node v_1 is a seed node. If this does not hold, one can label the seed node such that v_1 is the seed node. Starting from each node, the random walker visits node v_1 within two steps is at least

$$1 - \min\{1 - \alpha - \gamma, \alpha + \beta\} \frac{\alpha}{|S|}.$$

This implies that each element of the first column of P^2 is at least

$$1 - \min\{1 - \alpha - \gamma, \alpha + \beta\} \frac{\alpha}{|S|}.$$

Note that s^* is the stationary distribution. This means that

$$s^* P^2 = s^*.$$

With a similar derivation as the method in the link¹, we have that

$$\|s^{(0)} P^2 - s^* P^2\|_1 \leq \left(1 - \min\{1 - \alpha - \gamma, \alpha + \beta\} \frac{\alpha}{|S|}\right) \times \|s^{(0)} - s^*\|_1.$$

Then it follows that

$$\begin{aligned} \|s^{(0)} P^{2t} - s^* P^{2t}\|_1 &= \|s^{(0)} P^{2t} - s^* P^{2t}\|_1 \\ &\leq \left[1 - \min\{1 - \alpha - \gamma, \alpha + \beta\} \frac{\alpha}{|S|}\right]^t \\ &\quad \times \|s^{(0)} - s^*\|_1. \end{aligned}$$

This proof is then complete. ■

Proof of Theorem 3: Note that matrix P is a $3N$ by $3N$ square matrix. The dimension of the vector $\hat{s}^{(\tau)}$ is $3N$. Hence the space complexity is $O(N^2)$. The complexity of each multiplication of the vector to the matrix P is $O(N^2)$. Hence, the computational complexity $O(N^2 t(\epsilon))$ ■

¹<http://www.columbia.edu/%7E7Eww2040/6711F13/lect1029.pdf>

Proof of Theorem 4: Note that the space complexity is determined by the friendship graph. Thus, the space complexity is $O(N^2)$. Normalizing a node has a computational complexity of $O(N)$. There are N nodes in total to normalize. Thus, the computational complexity is $O(N^2)$. ■

Proof of Theorem 5: Note that the graph $\mathcal{G}$ has at most $3N$ nodes. The number of edges is at most $3N \times 3N = 9N^2$. Thus, the space complexity of Algorithm 1 is $O(N^2)$. ■

7. CONCLUSION

This paper presents SybilScaRepAct which detects sybil nodes in OSNs with a high accuracy and a high scalability toward activities. We formulate a new graph model called ScaSAN to represent users and activities in an OSN. ScaSAN is highly scalable with respect to activities in the sense that the number of nodes is at most three times the number of users. This significantly reduces the space complexity of the model in [5], which has a number nodes linear in the number of users plus the number of activities. We develop a SybilScaRepAct algorithm to evaluate trust scores for users based a given tiny set of seed nodes (honest or sybil seeds) and ScaSAN. SybilScaRepAct is interpretable via the random walk paradigm and it uses an iterative algorithm to approximate the trust score. We prove the convergence rate of this iterative algorithm and derive the number of rounds needed to achieve a given approximation accuracy. Our SybilScaRepAct also mitigates one limitation of the algorithm of [5]. Experiments on a public dataset from Facebook show the superior performance of SybilSRA over the algorithm of [5] in terms of detection accuracy and computational efficiency.

Our work is applicable in the social networks, where the behavior pattern of sybil users evolves slowly. Our activity aggregation method is just simple counting of numbers, which is highly fast for incorporating new activity data. As long as the behavior pattern of sybil users evolves slow enough, the constructed graph is accurate, otherwise it would be outdated and misleading. One potential future direction is to exploring the dynamic setting, where behavior of sybils evolves rapidly across time and they may even game the detection algorithm [45]. One naive way is by adding a discounting factor in activity aggregation, paying more attention to recent activities than old activities. Exploring sophisticated methods is an interesting future work.

A. LIMITATIONS OF SYBILSOCACTNET

In this section, we reveal fundamental understanding on the algorithmic limitations of the typical method SybilSocActNet. This understanding also give us insights to reveal why directly applying SybilSocActNet to ScaSAN leads to low accuracy.

A.1. Understanding the Limitations of SybilSocActNet

Fig. 9 shows the transition probability of the inter random walk part of SybilSocActNet. In particular, this inter random walk walks between the user nodes and activity nodes in an interchanging manner, i.e., if the walker is at a user (or activity) node currently, then it jumps to an activity (or user) node in next move. Fig. 9 shows that when the walker jumps from activities a_1, a_2 or a_7 to user node v_2 , the walker will never leave user v_2 , as in the next move the walker jumps to an activity created by v_2 but v_2 does not create any activities. Namely, v_2 is a trapping node. As we increase k , the trapping effect will be enlarged. This explains the low accuracy of SybilSocActNet discussed in Section 1. One may suggest to select a small k , i.e., a small number of random walks across the user nodes and activity nodes. However, this would reduce the propagation of trust between users and activities and lead to a low accuracy on other graphs.

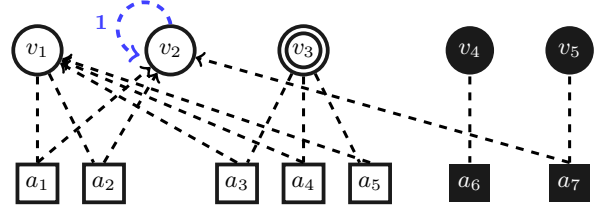


Fig. 9: The limitation of SybilSocActNet.

A.2. Limitations of Applying SybilSocActNet to ScaSAN

Now, we show that by directly applying SybilSocActNet to ScaSAN, the detection accuracy can also be very low. As shown in Fig. 10, applying the afore mentioned inter random walk part of SybilSocActNet described in Section A.1, v_2 is still a trapping node under ScaSAN. Fig. 11 shows that directly applying SybilSocActNet to ScaSAN (i.e., ignoring the weight of edges because SybilSocActNet considers unweighted graph), the detection accuracy can be as low as 0.5. This result further justifies that by directly applying SybilSocActNet to ScaSAN, the detection accuracy could also be very low.

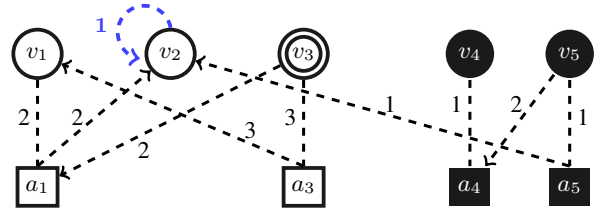


Fig. 10: Trapping state of applying SybilSocActNet to ScaSAN.

Summary: One limitation of SybilSocActNet is that walkers will be trapped by user nodes who do not create any activity. Such trapping nodes may lead to low accuracy of SybilSocActNet. Directly

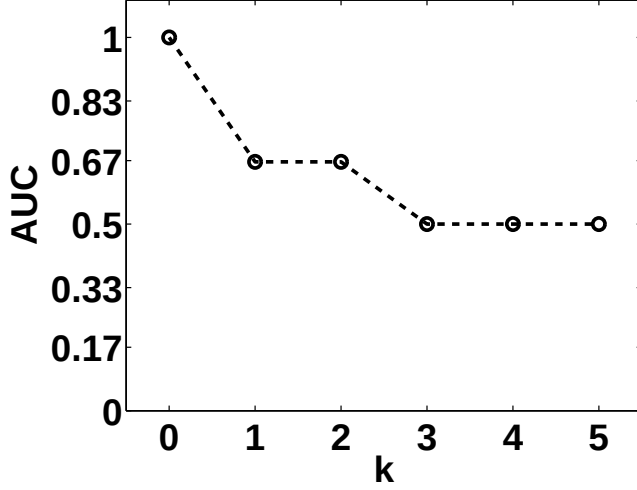


Fig. 11: Impact of k ($n = 1$) on Fig. 10.

applying SybilSocActNet to ScaSAN also has a low accuracy as the trapping node is an intrinsic feature of the underlying OSN and it still exists in ScaSAN.

B. ADDITIONAL EXPERIMENTS

Impact of outgoing activity attack. Fig. 12a shows the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship when aggressiveness of sybil nodes initiating activities to honest nodes ζ varies from 0.27 to 17.28. From Fig. 12a, one can observe that the AUC curve of SybilScaRepAct lies above that of SybilSocActNet. In other words, SybilScaRepAct has higher accuracy than SybilSocActNet. Furthermore, when ζ increases from 0.27 to 17.28, the AUC of both SybilScaRepAct and SybilSocActNet increases slightly. This implies that both these two algorithm can utilize outgoing interaction attack to detect sybil. The AUC curves of SybilFriendship are flat because this algorithm detect sybil using friendship edges only. The reason is that the trust score is propagated through the incoming edges. Increasing the number of outgoing attack edges would not make the sybil more distinguishable or less distinguishable with respect to trust propagation.

Impact of seed. Fig. 12b shows the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship when the number of seeds increases from 5 to 20. From Fig. 12b, one can observe that the AUC curve of SybilScaRepAct lies above that of SybilSocActNet. In other words, SybilScaRepAct has higher accuracy than SybilSocActNet. Furthermore, when the number of seeds increases from 5 to 20, the AUC of both SybilScaRepAct and SybilSocActNet increases slightly. This implies that both these two algorithms are not sensitive to the number of seeds. The reason is that the amount of sybil user is quite small compared to the number of honest users. The number of seed nodes is also very small compared to the number of honest nodes.

Impact of weight of activity edge. Fig. 13a and 13b show the impact of weight of activity edges, i.e., λ_1 and λ_2 , on the AUC of SybilScaRepAct, SybilSocActNet and SybilFriendship. From Fig. 13a and 13b, one can observe that the AUC curves of the SybilSocActNet and SybilFriendship are flat. The reason is that these two algorithms do not have the parameters λ_1 and λ_2 . One can also observe that the AUC curve of SybilScaRepAct is flat in the selected moderately small range of $\lambda_1 \in [1, 2.5]$ and $\lambda_2 \in [2.5, 4]$. This

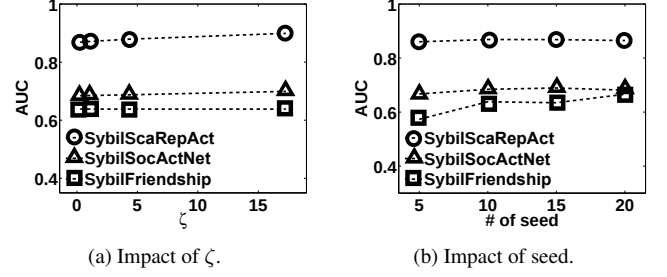


Fig. 12: Impact of outgoing activity attack and seed number.

implies that our SybilScaRepAct is not sensitive to the selection of λ_1 and λ_2 , which further justifies the stableness of our previous experiment results. The reason is that λ_1 and λ_2 in total control a small fraction of trust score, thus when they are in a moderately small range, the difference is small.

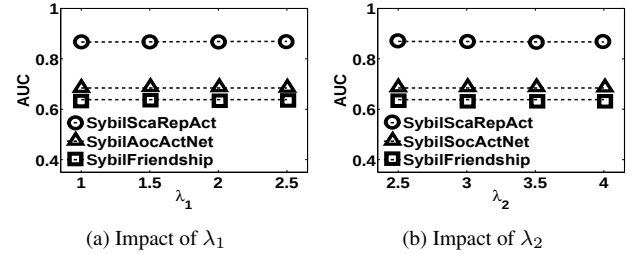


Fig. 13: Impact of weight of activity edge.

Summary. The algorithm is insensitive to outgoing attack, since the trust score propagates through incoming attack edges. The algorithm is insensitive to seeds, when the number of seeds is small. The algorithm is insensitive to λ_1 and λ_2 , when they are of a moderately small range.

C. REFERENCES

- [1] Haifeng Yu, Phillip B Gibbons, Michael Kaminsky, and Feng Xiao, "Sybillimit: A near-optimal social network defense against sybil attacks," in *IEEE S&P*, 2008.
- [2] Haifeng Yu, Michael Kaminsky, Phillip B Gibbons, and Abraham Flaxman, "Sybilguard: defending against sybil attacks via social networks," in *ACM CCR*. ACM, 2006, vol. 36, pp. 267–278.
- [3] Jinyuan Jia, Binghui Wang, and Neil Zhenqiang Gong, "Random walk based fake account detection in online social networks," in *IEEE DSN*, 2017.
- [4] Binghui Wang, Le Zhang, and Neil Zhenqiang Gong, "Sybilscar: Sybil detection in online social networks via local rule based propagation," in *IEEE INFOCOM*, 2017.
- [5] Xiaoying Zhang, Hong Xie, Pei Yi, and John C.S. Lui, "Enhancing sybil detection via social-activity networks: A random walk approach," *IEEE TDSC*, vol. 20, no. 2, pp. 1213–1227, 2023.
- [6] Qiang Cao, Michael Sirivianos, Xiaowei Yang, and Tiago Pregueiro, "Aiding the detection of fake accounts in large scale social online services," in *USENIX NSDI*, 2012.
- [7] G Danezis and P Mittal, "Sybilinifer: Detecting sybil nodes using social networks,," in *USENIX NDSS*, 2009.
- [8] Neil Zhenqiang Gong, Mario Frank, and Prateek Mittal, "Sybilbelief: A semi-supervised learning approach for structure-based sybil detection," *IEEE TIFS*, vol. 9, no. 6, pp. 976–987, 2014.
- [9] Nguyen Tran, Jinyang Li, Lakshminarayanan Subramanian, and Sherman SM Chow, "Optimal sybil-resilient node admission control," in *IEEE INFOCOM*, 2011.
- [10] Wei Wei, Fengyuan Xu, Chiu C Tan, and Qun Li, "Sybildefender: Defend against sybil attacks in large social networks," in *IEEE INFOCOM*, 2012.
- [11] Lorenzo Alvisi, Allen Clement, Alessandro Epasto, Silvio Lattanzi, and Alessandro Panconesi, "Sok: The evolution of sybil defense via social networks," in *IEEE S&P*, 2013.
- [12] Zhi Yang, Christo Wilson, Xiao Wang, Tingting Gao, Ben Y Zhao, and Yafei Dai, "Uncovering social network sybils in the wild," *ACM TKDD*, vol. 8, no. 1, pp. 2, 2014.
- [13] Vasumathi Sridharan, Vaibhav Shankar, and Minaxi Gupta, "Twitter games: How successful spammers pick targets," in *ACSAC*, 2012.
- [14] Yazan Boshmaf, Ildar Muslukhov, Konstantin Beznosov, and Matei Ripeanu, "The socialbot network: when bots socialize for fame and money," in *ACSAC*, 2011.
- [15] David Koll, Martin Schwarzaier, Jun Li, Xiang-Yang Li, and Xiaoming Fu, "Thank you for being a friend: An attacker view on online-social-network-based sybil defenses," in *IEEE ICDCSW*, 2017.
- [16] Suhendry Effendy and Roland HC Yap, "The strong link graph for enhancing sybil defenses," in *IEEE ICDCS*, 2017.
- [17] Xiao Liang, Zheng Yang, Binghui Wang, Shaofeng Hu, Zijie Yang, Dong Yuan, Neil Zhenqiang Gong, Qi Li, and Fang He, "Unveiling fake accounts at the time of registration: An unsupervised approach," in *ACM SIGKDD*, 2021.
- [18] Zheng Qu, Chen Lyu, and Chi-Hung Chi, "Mush: Multi-stimuli hawkes process based sybil attacker detector for user-review social networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4600–4614, 2022.
- [19] Siyu Li, Jin Yang, Gang Liang, Tianrui Li, and Kui Zhao, "Sybilflyover: Heterogeneous graph-based fake account detection model on social networks," *Knowledge-Based Systems*, vol. 258, pp. 110038, 2022.
- [20] Haitao Huang, Hu Tian, Xiaolong Zheng, Xingwei Zhang, Daniel Dajun Zeng, and Fei-Yue Wang, "Cgnn: A compatibility-aware graph neural network for social media bot detection," *IEEE Transactions on Computational Social Systems*, vol. 11, no. 5, pp. 6528–6543, 2024.
- [21] Greeshma Lingam, Rashmi Ranjan Rout, Durvasula VLN Somayajulu, and Sajal K Das, "Social botnet community detection: a novel approach based on behavioral similarity in twitter network using deep learning," in *ACM AsiaCCS*, 2020.
- [22] Qiang Cao, Xiaowei Yang, Jieqi Yu, and Christopher Palow, "Uncovering large groups of active malicious accounts in online social networks," in *ACM CCS*, 2014.
- [23] Stefano Cresci, Roberto Di Pietro, Marinella Petrocchi, Angelo Spognardi, and Maurizio Tesconi, "Dna-inspired online behavioral modeling and its application to spambot detection," *IEEE Intelligent Systems*, vol. 31, no. 5, pp. 58–64, 2016.
- [24] Zachary Miller, Brian Dickinson, William Deitrick, Wei Hu, and Alex Hai Wang, "Twitter spammer detection using data stream clustering," *Information Sciences*, vol. 260, pp. 64–73, 2014.
- [25] Gang Wang, Tristan Konolige, Christo Wilson, Xiao Wang, Haitao Zheng, and Ben Y Zhao, "You are how you click: Clickstream analysis for sybil detection,," .
- [26] Gianluca Stringhini, Christopher Kruegel, and Giovanni Vigna, "Detecting spammers on social networks," in *ACSAC*, 2010.
- [27] Zenghua Xia, Chang Liu, Neil Zhenqiang Gong, Qi Li, Yong Cui, and Dawn Song, "Characterizing and detecting malicious accounts in privacy-centric mobile social networks: A case study," in *ACM SIGKDD*, 2019.
- [28] Fred Morstatter, Do Own Kim, Natalie Jonckheere, Calvin Liu, Malika Seth, and Dmitri Williams, "I'll play on my other account: The network and behavioral differences of sybils," *ACM HCI*, 2021.
- [29] Xiaojie Xu, Jian Dong, Zhengyu Liu, Jin Yang, Bin Wang, and Zhaoyuan Wang, "A sybil detection method in osn based on distilbert and double-sn-lstm for text analysis," in *IEEE SecureComm*, 2021, pp. 64–76.
- [30] Siyu Li, Jin Yang, and Kui Zhao, "Are you in a masquerade? exploring the behavior and impact of large language model driven social bots in online social networks," *arXiv preprint arXiv:2307.10337*, 2023.
- [31] Hongling Jiang, Dan Liu, Haiyan Kang, and Yilin Wang, "A fusion malicious social bots detection model based on static and dynamic features," *International Journal of Network Security*, vol. 24, no. 2, pp. 321–332, 2022.
- [32] Yulin Liu, "Quantum-based detection of highly semantically similar social bot," *Frontiers in Computing and Intelligent Systems*, vol. 3, no. 3, pp. 38–42, 2023.

- [33] Satoshi Furutani, Toshiki Shibahara, Kunio Hato, Mitsuaki Akiyama, and Masaki Aida, "Sybil detection as graph filtering," in *IEEE GLOBECOM*, 2020.
- [34] Yazan Boshmaf, Dionysios Logothetis, Georgos Siganos, Jorge Lería, Jose Lorenzo, Matei Ripeanu, Konstantin Beznosov, and Hassan Halawa, "Íntegro: Leveraging victim prediction for robust fake account detection in large scale osns," *Computers & Security*, vol. 61, pp. 142–168, 2016.
- [35] János Höner, Shinichi Nakajima, Alexander Bauer, Klaus-Robert Müller, and Nico Görnitz, "Minimizing trust leaks for robust sybil detection," in *ICML*, 2017.
- [36] Qiang Cao, Michael Sirivianos, Xiaowei Yang, and Kamesh Munagala, "Combating friend spam using social rejections," in *IEEE ICDCS*, 2015.
- [37] Jilong Xue, Zhi Yang, Xiaoyong Yang, Xiao Wang, Lijiang Chen, and Yafei Dai, "Votetrust: Leveraging friend invitation graph to defend against social network sybils," in *IEEE INFOCOM*, 2013.
- [38] Peng Gao, Binghui Wang, Neil Zhenqiang Gong, Sanjeev R Kulkarni, Kurt Thomas, and Prateek Mittal, "Sybilfuse: Combining local attributes with global structure to perform robust sybil detection," in *IEEE CNS*, 2018.
- [39] Feng Liu, Chunfang Yang, Zhenyu Li, Daofu Gong, Rui Ma, and Fenlin Liu, "Accou2vec: A social bot detection model based on community walk," *IEEE TDSC*, pp. 1–17, 2023.
- [40] Mingzhou Yang, Xingwei Wang, Min Huang, Lianbo Ma, and Qiang He, "A novel evolutionary algorithm based on judgment-rule evolution strategy for structural balance in signed social networks," *IEEE TETCI*, vol. 6, no. 4, pp. 805–817, 2022.
- [41] Badis Hammi, Yacine Mohamed Idir, Sherali Zeadally, Rida Khatoun, and Jamel Nebhen, "Is it really easy to detect sybil attacks in c-its environments: A position paper," *IEEE TITS*, vol. 23, no. 10, pp. 18273–18287, 2022.
- [42] Zhi Yang, Yusi Zhang, and Yafei Dai, "Defending against social network sybils with interaction graph embedding," in *IEEE CNS*, 2018.
- [43] Dong Yuan, Yuanli Miao, Neil Zhenqiang Gong, Zheng Yang, Qi Li, Dawn Song, Qian Wang, and Xiao Liang, "Detecting fake accounts in online social networks at the time of registrations," in *ACM CCS*, 2019.
- [44] Zheng Liu, Yu Weng, Ruiyang Xu, Chaomurilige, and Honghao Gao, "A hierarchical information compression approach for knowledge discovery from social multimedia," *IEEE TCSS*, vol. 11, no. 6, pp. 7754–7765, 2024.
- [45] Dongwon Shin, Suyoung Lee, and Soeul Son, "Ricc: Robust collective classification of sybil accounts," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2329–2339.
- [46] Lianbo Ma, Haidong Kang, Guo Yu, Qing Li, and Qiang He, "Single-domain generalized predictor for neural architecture search system," *IEEE TC*, vol. 73, no. 5, pp. 1400–1413, 2024.
- [47] Ye Wang, Honghao Gao, Zhengzhe Xiang, Zhongzhi Zhu, and Anwer Al-Dulaimi, "Rsmr: A reliable and sustainable multi-path routing scheme for vehicle electronics in edge computing networks," *IEEE TCE*, pp. 1–1, 2024.
- [48] Hongwei Zeng, Zhongzhi Zhu, Ye Wang, Zhengzhe Xiang, and Honghao Gao, "Periodic collaboration and real-time dispatch using an actor-critic framework for uav movement in mobile edge computing," *IEEE IOT*, vol. 11, no. 12, pp. 21215–21226, 2024.
- [49] Honghao Gao, Xuejie Wang, Wei Wei, Anwer Al-Dulaimi, and Yueshen Xu, "Com-ddpg: Task offloading based on multi-agent reinforcement learning for information-communication-enhanced mobile edge computing in the internet of vehicles," *IEEE TVT*, vol. 73, no. 1, pp. 348–361, 2024.
- [50] Bimal Viswanath, Alan Mislove, Meeyoung Cha, and Krishna P Gummadi, "On the evolution of user interaction in facebook," in *ACM OSN*, 2009.
- [51] Jinxue Zhang, Rui Zhang, Jingchao Sun, Yanchao Zhang, and Chi Zhang, "Truetop: A sybil-resilient system for user influence measurement on twitter," *IEEE/ACM ToN*, vol. 24, no. 5, pp. 2834–2846, 2016.